TERRAFORM INTERVIEW CHEAT SHEET — 3 YOE DevOps

Keywords & technical terms only — quick-scan before the interview

CORE & LIFECYCLE

Terraform: HashiCorp IaC, declarative, multi-cloud, agentless
Lifecycle: Write → `init` → `validate` → `plan` → `apply` → `destroy`
Providers: plugins for AWS / Azure / GCP / K8s / Helm APIs

BLOCK TYPES

`terraform{}` version + backend, `provider{}` config
`resource{}` managed obj, `data{}` read-only fetch
`variable{}` input, `output{}` exposed value
`locals{}` internal value, `module{}` reusable call

CLI COMMANDS

`init` setup backend/providers · `validate` syntax check
`fmt -check` · `plan -out=` · `apply` -auto-approve
`destroy -target=` · `output` · `show`

RESOURCE META-ARGUMENTS

count: numeric, `count.index`
for_each: map/set, `each.key`/`each.value`
depends_on: explicit dep (implicit ref preferred)

LIFECYCLE BLOCK & TYPES

`create_before_destroy`, `prevent_destroy`
`ignore_changes` [attrs]/all,
`replace_triggered_by`
`precondition`/`postcondition` (1.2+)

DEPENDENCY GRAPH

DAG (Directed Acyclic Graph) of all resources
Implicit (attr ref, preferred) vs explicit (`depends_on`)
`terraform graph` → DOT format, Graphviz

VARIABLES / OUTPUTS / LOCALS

types: string, number, bool, list, map, object, tuple
precedence: default < env < .tfvars < auto.tfvars < -var-file < -var
output: `sensitive`, `depends_on` · locals = DRY values

PROVIDER CONFIG

`alias` for multi-region/account setups
`required_providers{}` version pin (~>, >=)
`.terraform.lock.hcl` commit it · `default_tags`

STATE MANAGEMENT

tfstate = JSON map: config → real resource IDs
local vs remote (remote = team single source of truth)
never commit state to Git; may hold secrets in plaintext

STATE FILE STRATEGY

1 state per **environment** → blast-radius control
1 state per **layer/module** (network/platform/app)
key pattern: `env/component/terraform.tfstate`

REMOTE BACKENDS

AWS: S3 (storage) + DynamoDB (lock table)
Azure: azurerm → Storage Account+Blob · GCP: GCS · TFC/E built-in
`backend{}` block: no vars/interpolation allowed

CROSS-STATE / REMOTE STATE DATA SOURCE

`terraform_remote_state` — reads another state's outputs
use case: network team's VPC/subnet ID → consumed by app team
decouples state ownership per team while allowing composition

MODULE COMMUNICATION & EXAMPLE

root wires modules: `module.vpc.subnet_id` → input of next module
cross-root: use `terraform_remote_state`
`module "vpc" { source="./modules/vpc" }`

MULTI-ENV STRATEGY

Workspaces: `terraform workspace new/select`, light isolation
Env dirs: environments/dev,stage,prod — own backend+tfvars
Env-dir preferred in orgs: full isolation, no accidental cross-env apply

REPO STRUCTURE

`modules/` reusable blocks (vpc, ec2, rds)
`environments/<env>/` main.tf, backend.tf, tfvars
`global/` account-level (IAM, DNS) · versions.tf per module

DATA SOURCES

read-only fetch of existing/external infra, no lifecycle mgmt
e.g. `data "aws_ami"` / `data "aws_vpc"`

EXPRESSIONS & FUNCTIONS

ternary: `cond ? a : b` · `for` expr, `dynamic{}` blocks
fns: `lookup()`, `merge()`, `join()`, `coalesce()`
`templatefile()`, `jsonencode()`, `try()`

STATE DRIFT & IGNORE_CHANGES

drift = real infra ≠ state; caught by `plan`
`ignore_changes = [attr]` or `all` — silence expected drift
`apply -refresh-only` — sync state without changing infra

STATE CMDS — FREQUENTLY USED

`state list` · `state show <addr>`
`state mv <old> <new>` · `state rm <addr>`
`state pull` / `push` — raw state backup/restore

IMPORT / TAINT / REPLACE

`terraform import <addr> <id>` — needs matching resource block
taint deprecated → `apply -replace=<addr>`
import doesn't generate HCL — write config manually

STATE RECOVERY & DR CHECKLIST

`state pull > backup.tfstate` before risky ops
S3 versioning + SSE, DynamoDB PITR, cross-region replica
no backup → rebuild via `import` per resource; test DR runbook

STATE LOCKING & LOCK ERRORS

DynamoDB/native lock — blocks concurrent plan/apply
error: check LockID/Who/Created → confirm no active run
`force-unlock <ID>` only after confirming it's safe

PROVISIONERS

`local-exec` / `remote-exec` — last resort only
creation-time vs `when = destroy`
`connection{}` block — SSH/WinRM

SECRETS IN TERRAFORM

never hardcode; use Vault / SSM / Key Vault
`sensitive = true` masks CLI output (still plaintext in state)
.gitignore: `*.tfstate`, `*.tfvars`, `.terraform/`

TAGS / TAGGING STRATEGY

`default_tags{}` (AWS provider) — auto-applies to all resources
standard set: Environment, Owner, Project, CostCenter

TESTING & VALIDATION

`validate`, `fmt -check` (CI gate)
`tflint` lint · `checkov`/`tfsec` security scan
`plan -detailed-exitcode` (0=none,1=err, 2=changes)

CI/CD INTEGRATION

fmt → validate → plan → manual approval → apply
plan output as PR artifact/comment before apply
remote backend locking across pipeline runs

TERRAFORM CLOUD/ENTERPRISE

remote runs, VCS-driven, built-in state + locking
Sentinel/OPA — policy-as-code gates
private registry, RBAC, run triggers

COMMON ERRORS

provider conflict → pin version, delete .terraform & re-init
cyclic dependency → refactor refs/`depends_on`
"already exists" → `import` instead of recreate

BEST PRACTICES

small composable modules; remote state + locking always
consistent naming/tagging; pin provider/module versions
never edit state manually; review plan before every apply